Session

Session คือกลไกที่ทำให้เว็บไซต์จดจำข้อมูลของผู้ใช้ข้ามคำขอ (request) ได้ เช่น จำว่าล็อกอินแล้วเป็นใคร หรือมีสินค้าอะไรในตะกร้าบ้าง โดยปกติ HTTP ไม่ได้จดจำสถานะของคำขอก่อนหน้า เราจึงใช้ session เชื่อมคำขอหลายครั้งให้เป็นการใช้งานของผู้ใช้คนเดียวกัน

1. Session ในโค้ดนี้ทำงานอย่างไร

โค้ดนี้ใช้แพ็กเกจ express-session ซึ่งแบ่งข้อมูลออกเป็นสองส่วน:

ส่วนประกอบ	**อยู่ที่ไหน**	**หน้าที่**
Session ID	อยู่ใน cookie ของ browser	เป็นรหัสสำหรับอ้างอิง session
Session data	อยู่บน server ใน session store	เก็บข้อมูล เช่น userId หรือตะกร้าสินค้า
Session middleware	อยู่ใน Express	อ่าน cookie โหลดข้อมูล และทำให้เราใช้ req.session ได้

Browser เก็บรหัสอ้างอิง ส่วน server เก็บข้อมูล session โดยชื่อ cookie เริ่มต้นคือ connect.sid โค้ดที่ให้มายังไม่ได้ระบุ store จึงใช้ MemoryStore ซึ่งเก็บข้อมูลในหน่วยความจำของโปรเซส Node.js โดยอัตโนมัติ เหมาะสำหรับฝึกและพัฒนา แต่ระบบใช้งานจริงควรเปลี่ยนเป็น store เช่น PostgreSQL หรือ Redis

สมมติผู้ใช้ล็อกอินสำเร็จ

1. Server บันทึก userId: 15 ลงใน session
2. Server ส่ง cookie ที่มี Session ID กลับไปให้ browser
3. เมื่อ browser ขอหน้า /profile จะส่ง cookie กลับมาอัตโนมัติ
4. Middleware ใช้ Session ID ค้นข้อมูลบน server
5. โค้ดใน route อ่าน req.session.userId แล้วรู้ว่าเป็นผู้ใช้หมายเลข 15

ตัวอย่างข้อมูลบน server แบบย่อ:

```
// Session ID: abc123
{
  userId: 15,
  cart: [
    { productId: 8, quantity: 2 }
  ]
}
```

ข้อมูล userId และ cart เหล่านี้ไม่ได้ถูกใส่ลงใน session cookie ของ express-session

2. อธิบายแต่ละส่วนของโค้ด

```
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    sameSite: 'lax',
    maxAge: 60 * 60 * 1000
  }
}));
```

app.use(...) — ติดตั้ง middleware

ให้ Express ประมวลผล session ก่อนเข้า route ที่ประกาศไว้ด้านล่าง

```
app.use(session({ /* ตัวเลือก */ }));

// Route นี้จึงใช้งาน req.session ได้
app.get('/profile', (req, res) => {
  res.json({ userId: req.session.userId });
```

});

ต้องวาง session middleware **ก่อน route ที่ต้องใช้ session**

secret — กุญแจสำหรับลงลายเซ็น cookie

secret: process.env.SESSION_SECRET

ใช้ลงลายเซ็น Session ID เพื่อให้ server ตรวจพบการแก้ไขค่า cookie โดยมิชอบ **ไม่ได้ใช้เข้ารหัสข้อมูล session หรือเข้ารหัสรหัสผ่าน** process.env.SESSION_SECRET คืออ่านค่าจาก environment variable โดยควรใช้ค่าที่สุ่มและคาดเดายาก คงค่าเดิมไว้ระหว่างการรีสตาร์ต และใช้ค่าเดียวกันเมื่อมีหลาย server ให้บริการแอปเดียวกัน ถ้าใช้ไฟล์ .env สามารถโหลดด้วย dotenv ซึ่งในไฟล์ .env ต้องมีรายการนี้

SESSION_SECRET=ใส่ค่าสุ่มที่ยาวและคาดเดายาก

และเวลาเรียกใช้งาน .env ต้องใช้คำสั่งนี้ในไฟล์ที่ต้องการ

require('dotenv').config();

สร้างค่าสุ่มด้วยคำสั่งนี้ แล้วนำผลลัพธ์ไปใส่ใน .env:

node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

resave: false — ไม่บันทึกข้อมูลเดิมซ้ำโดยไม่มีการเปลี่ยนแปลง

เปรียบเทียบสองกรณี:

// อ่านอย่างเดียว ไม่ได้แก้ข้อมูล session

console.log(req.session.userId);

// แก้ข้อมูล session จึงต้องบันทึก

req.session.userId = 15;

false ไม่ได้หมายความว่าห้ามบันทึก แต่หมายถึงไม่บังคับเขียนข้อมูล session เดิมซ้ำทุก request ทั้งนี้ store อาจยังอัปเดตเวลาหมดอายุผ่านเมธอด touch ได้ และควรตรวจว่า store ที่เลือกสนับสนุนพฤติกรรมนี้

saveUninitialized: false — ไม่บันทึก session ใหม่ที่ยังไม่ได้ใช้งาน

ถ้าผู้ใช้เปิดหน้าแรก แต่เราไม่ได้ใส่ข้อมูลใด ๆ ลงใน session ก็ยังไม่ต้องบันทึก session หรือส่ง session cookie

เมื่อมีการกำหนดข้อมูล เช่น

req.session.cart = [];

session นั้นถือว่าถูกแก้ไขแล้ว จึงบันทึกได้ **ไม่จำเป็นต้องรอให้ล็อกอินก่อน** ตัวเลือกนี้เหมาะเมื่อไม่ต้องการสร้าง session ให้ผู้เยี่ยมชมทุกคนทันที

ความต่างของสองตัวเลือกนี้คือ

ตัวเลือก	**คำถามที่ควบคุม**
saveUninitialized	session ใหม่ที่ยังไม่ได้แก้ข้อมูล ต้องบันทึกไหม?
resave	session ที่มีอยู่แล้วและข้อมูลไม่เปลี่ยน ต้องบันทึกซ้ำไหม?

cookie — กำหนดพฤติกรรมของ cookie ที่เก็บ Session ID

ตัวเลือก	**ความหมาย**	**เหตุผลที่ใช้**
httpOnly: true	JavaScript ในหน้าเว็บอ่าน cookie นี้ผ่าน document.cookie ไม่ได้	ลดโอกาสที่สคริปต์จะอ่าน Session ID
sameSite: 'lax'	จำกัดการส่ง cookie ในคำขอข้ามเว็บไซต์ แต่ยังส่งได้เมื่อผู้ใช้คลิกลิงก์แบบ GET เข้ามา	เหมาะกับเว็บไซต์ทั่วไป และช่วยลดความเสี่ยง CSRF บางกรณี
maxAge: 60 * 60 * 1000	กำหนดอายุ cookie เป็นมิลลิวินาที	เท่ากับ 1 ชั่วโมง

Browser ยังส่ง cookie ที่เป็น httpOnly กลับ server ตามปกติ และ sameSite: 'lax' ไม่ใช่การป้องกัน CSRF ที่ครอบคลุมทุกกรณี

การคำนวณเวลา

60 * 60 * 1000

// นาที × วินาที × มิลลิวินาที

// = 3,600,000 มิลลิวินาที = 1 ชั่วโมง

จุดที่มักเข้าใจผิด: maxAge ไม่ได้แปลว่า “ล็อกอินแล้วออกจากระบบแน่นอนเมื่อครบ 1 ชั่วโมง” เพราะอายุ cookie อาจถูกต่อเมื่อมีการแก้ไข session ถ้าต้องการต่ออายุ cookie ทุก response ของ session ที่มีอยู่แล้ว ให้เพิ่ม

rolling: true

จึงเหมาะกับพฤติกรรม “ยังใช้งานต่อเนื่องก็อยู่ในระบบได้ หากไม่มี request เป็นเวลา 1 ชั่วโมง cookie จึงหมดอายุ” ส่วนเวลาหมดอายุของข้อมูลฝั่ง server ต้องสอดคล้องกับการตั้งค่าของ store ด้วย

3. ทดลองใช้จริงด้วยตัวนับจำนวนครั้ง

ตัวอย่างนี้ช่วยให้เห็นว่า session จำข้อมูลข้าม request อย่างไร

ติดตั้ง

npm init -y

npm install express express-session dotenv

สร้าง .env ใส่ SESSION_SECRET ตามตัวอย่างข้างต้น แล้วสร้าง app.js:

```
require('dotenv').config();

const express = require('express');
const session = require('express-session');

const app = express();

if (!process.env.SESSION_SECRET) {
  throw new Error('กรุณากำหนด SESSION_SECRET');
}

app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
```

```
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    sameSite: 'lax',
    maxAge: 60 * 60 * 1000
  }
}));

// อ่าน session โดยไม่แก้ไขข้อมูล
app.get('/read', (req, res) => {
  res.json({
    views: req.session.views ?? 0
  });
});

// เพิ่มตัวนับใน session
app.get('/count', (req, res) => {
  req.session.views = (req.session.views ?? 0) + 1;

  res.json({
    views: req.session.views
  });
});

// ล้าง session
app.post('/reset', (req, res, next) => {
  req.session.destroy((err) => {
    if (err) return next(err);
```

```
    res.clearCookie('connect.sid', { path: '/' });
    res.json({ message: 'ล้าง session แล้ว' });
  });
});

app.listen(3000, () => {
  console.log('เปิด http://localhost:3000');
});
```

รัน:

```
node app.js
```

ทดลองตามลำดับ

สิ่งที่ทำ	**ผลที่ควรเห็น**
เปิด /read ครั้งแรก	views: 0 และยังไม่มี session cookie หากยังไม่เคยใช้งาน
เปิด /count	views: 1 และได้รับ session cookie
รีเฟรช /count	เพิ่มเป็น 2, 3, 4
เปิด /read	อ่านค่าล่าสุด แต่ไม่เพิ่มตัวนับ
เปิด /count ในหน้าต่างไม่ระบุตัวตน	เริ่มที่ 1 เพราะมีชุด cookie แยกกัน
รีสตาร์ต Node.js แล้วเปิด /count	เริ่มใหม่ เพราะตัวอย่างใช้ MemoryStore

ดู cookie ได้ที่ **DevTools ⟶ Application ⟶ Cookies ⟶ http://localhost:3000** จะพบ connect.sid หลังเปิด /count

4. ใช้ session กับระบบล็อกอินอย่างไร

หลังตรวจ username และ password กับฐานข้อมูลสำเร็จแล้ว จึงบันทึกตัวตนลง session โดยสร้าง Session ID ใหม่ก่อน

```
// user คือข้อมูลผู้ใช้ที่ผ่านการตรวจรหัสผ่านแล้ว
req.session.regenerate((err) => {
  if (err) return next(err);

  req.session.userId = user.id;

  req.session.save((err) => {
    if (err) return next(err);

    res.redirect('/profile');
  });
});
```

การ regenerate() เปลี่ยน Session ID หลังล็อกอิน ส่วน save() ในตัวอย่างช่วยให้บันทึกเสร็จก่อน redirect ปกติเมื่อแก้ข้อมูล session middleware จะบันทึกให้เมื่อจบ response อยู่แล้ว
จากนั้นสร้าง middleware ตรวจการล็อกอิน

```
function requireLogin(req, res, next) {
  if (!req.session.userId) {
    return res.status(401).send('กรุณาล็อกอินก่อน');
  }

  next();
}

app.get('/profile', requireLogin, (req, res) => {
```

```
  res.json({
    userId: req.session.userId
  });
});
```

Session ช่วยจำตัวตน ส่วนการตรวจสิทธิ์ต้องเขียนเพิ่ม เช่น นำ userId ไปอ่าน role และ permission จากฐานข้อมูลก่อนอนุญาตให้ลบข้อมูล

5. ใช้งานในโอกาสไหนได้บ้าง

งาน	**ตัวอย่างข้อมูลใน session**
จำผู้ใช้ที่ล็อกอิน	req.session.userId = 15
ตะกร้าสินค้าก่อนล็อกอิน	req.session.cart = [...]
แบบฟอร์มหลายขั้นตอน	req.session.formDraft = {...}
ข้อความแจ้งผลหลัง redirect	req.session.message = 'บันทึกสำเร็จ'
เก็บสถานะชั่วคราว	ขั้นตอนที่ทำอยู่ หรือภาษาที่เลือก

ข้อมูลถาวร เช่น คำสั่งซื้อที่ยืนยันแล้ว ควรบันทึกในฐานข้อมูลหลัก เพราะ session สามารถหมดอายุหรือถูกล้างได้

สำหรับระบบจริงที่ใช้ HTTPS ให้เพิ่ม cookie.secure: true และกำหนด trust proxy ให้ตรงกับโครงสร้างที่ติดตั้ง หากมี reverse proxy อยู่หน้า Express